

APhEx 9, 2014 (ed. Vera Tripodi)
Ricevuto il: 24/04/2013
Accettato il: 29/10/2013
Redattore: Francesca Ervas

T E M I

TEORIA DEGLI AUTOMI PER LINGUAGGI FORMALI

di Alessandro Aldini

ABSTRACT – Lo studio dei meccanismi del cervello umano deputati alla comprensione del linguaggio naturale, l'analisi del sequenziamento del genoma umano, la necessità per un calcolatore di interpretare un insieme di comandi. Sono situazioni apparentemente lontane tra loro, ma accomunate da uno stesso problema, ovvero l'esigenza di descrivere proprietà di sequenze di simboli, che possono rappresentare vocaboli, molecole, istruzioni di un linguaggio di programmazione e altro ancora. In questo ambito, il Novecento è stato teatro di studi complementari che hanno dato origine alla teoria dei linguaggi formali, come nel caso delle grammatiche di Chomsky, e degli automi riconoscitori, come nel caso delle macchine astratte di Kleene e di Turing. L'obiettivo di questo lavoro è introdurre in chiave storica, critica e scientifica gli elementi fondazionali della teoria degli automi riconoscitori di linguaggi formali.

1. INTRODUZIONE
 2. SU STRINGHE, STELLE E NASTRI
 3. ESPRESSIONI REGOLARI E AUTOMI A STATI FINITI
 4. LINGUAGGI LIBERI E AUTOMI A PILA
 5. VERSO I CONFINI DEL CALCOLABILE
 6. NOTE CONCLUSIVE
- BIBLIOGRAFIA

1. INTRODUZIONE

L'esigenza di caratterizzare proprietà di sequenze di simboli è un problema comune ad un'ampia varietà di settori, dalla linguistica alla biologia computazionale, passando per molteplici campi dell'informatica, dall'interpretazione del software da parte del calcolatore alla verifica di correttezza dei sistemi di calcolo e reti di computer. Una tale varietà nasce dall'interpretazione che si può dare dei simboli, che possono rappresentare termini di un linguaggio naturale, molecole, istruzioni di un linguaggio di programmazione, segnali digitali, messaggi di un protocollo di comunicazione o altri eventi.

Lo studio rigoroso di questo problema generale può essere affrontato da due punti di vista che vedremo essere ortogonali. Da una parte, possiamo caratterizzare le sequenze di simboli come espressioni di un linguaggio generate attraverso le regole di una grammatica formale. Dall'altra, possiamo immaginare di definire una macchina astratta, comunemente detta *automa*, che ha l'obiettivo di riconoscere il linguaggio descritto da tali sequenze. Per via della vastità di questi argomenti, scopo del presente lavoro è introdurre il secondo di questi approcci formali, senza perdere di vista il legame forte che esiste con il primo.

La teoria dei linguaggi formali e la teoria degli automi, sebbene sviluppatesi in contesti differenti, affondano le loro origini tra i lavori di grandi studiosi del Novecento, da Stephen Kleene ad Alan Turing, da Emil Post a Noam Chomsky. Si può anzi affermare che radici comuni risalgano all'opera di David Hilbert (cfr. Hilbert [1928]) ed al suo metodo assiomatico mirato a meccanizzare il procedimento dimostrativo di verità matematiche, metodo destinato a fallire ma che ha avuto il grande merito di impostare le problematiche alla base dei più significativi risultati di fondamenti della matematica del secolo scorso.

I linguaggi formali nascono dalle notazioni simboliche utilizzate per descrivere in modo rigoroso formalismi matematici, esemplare è il caso della logica dei predicati descritta da Frege [1893]. In effetti, il primo impulso alla teoria dei linguaggi formali viene dalla logica simbolica e, a partire da Thue [1906], dalla teoria della combinatoria delle parole. Lo slancio definitivo arriva invece negli anni '50 del secolo scorso, con gli studi di Chomsky sulle grammatiche a struttura di frase, di Schützenberger in ambito algebrico-combinatorio e di Kleene e altri sugli automi.

In particolare, Chomsky [1957] modella aspetti del linguaggio naturale per mezzo di grammatiche formali, asserendo che queste possano spiegare l'abilità dell'essere umano di produrre ed interpretare un numero infinito di frasi usando gli ingredienti finiti delle grammatiche. Tali ingredienti sono, essenzialmente, i simboli dell'alfabeto di riferimento per il linguaggio e le regole grammaticali, usate per combinare i simboli a formare espressioni ben formate del linguaggio.

L'approccio linguistico seguito da Chomsky è chiaramente per natura *generativo*, in quanto mira a descrivere un oggetto matematico, la grammatica, in grado di generare o costruire le frasi di un certo linguaggio. L'approccio computazionale basato su automi è invece per natura *riconoscitivo*, in quanto impiega una procedura automatica, o algoritmo, per il riconoscimento di tutte e sole le espressioni appartenenti al linguaggio di riferimento.

In generale, la teoria degli automi è stata fondamentale nell'ambito della meccanizzazione dei processi di ragionamento che ha tenuto banco tra i matematici del Novecento. I risultati più visibili sono la definizione della macchina astratta di Turing e gli automi a stati finiti di Kleene, strumenti rivelatisi fondamentali per comprendere concetti all'epoca

ampiamente in discussione, come la calcolabilità (come caratterizzare i problemi risolvibili attraverso un algoritmo) e la complessità (quante risorse sono necessarie per effettuare i relativi calcoli).

Nel seguito, dopo aver fissato alcuni concetti formali di base, vengono discusse le classi di automi sviluppatesi storicamente a partire dalla Macchina di Turing (MdT), la loro espressività e la non sorprendente relazione con le grammatiche a struttura di frase di Chomsky.

2. SU STRINGHE, STELLE E NASTRI

Per definire formalmente un linguaggio occorre fissare un alfabeto Σ composto di un insieme finito di simboli che vengono combinati in sequenza a formare stringhe attraverso l'operazione di base di *concatenazione* di simboli. L'insieme finito di tutte le possibili stringhe di lunghezza k è definito come segue:

$$\Sigma^k = \{a_1 \cdots a_k \mid a_1, \dots, a_k \in \Sigma\}$$

Ad esempio, $abba \in \Sigma^4$ è una stringa di lunghezza 4 definita su un alfabeto Σ che include i simboli a e b . L'insieme infinito di stringhe di qualunque lunghezza è:

$$\bigcup_{i=0}^{\infty} \Sigma^i$$

e prende il nome di chiusura di Kleene di Σ , denotato Σ^* (l'operatore $*$ è noto come stella di Kleene). Si noti che Σ^* include la stringa di lunghezza zero, ovvero la stringa vuota,

comunemente indicata con il simbolo ϵ .

Un linguaggio formale L sull'alfabeto Σ è un qualunque sottoinsieme della chiusura di Kleene di Σ , $L \subseteq \Sigma^*$. Essendo definiti come insiemi di stringhe, possiamo applicare ai linguaggi formali le usuali operazioni della teoria degli insiemi. Ad esempio, l'unione di linguaggi è definita come $L_1 \cup L_2 = \{w \mid w \in L_1 \vee w \in L_2\}$ e l'intersezione come $L_1 \cap L_2 = \{w \mid w \in L_1 \wedge w \in L_2\}$. Una operazione ricorrente è la concatenazione di linguaggi, $L_1 L_2 = \{w_1 w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2\}$, che a sua volta è alla base della definizione di chiusura di Kleene di un linguaggio, $L^* = \{\epsilon\} \cup L \cup LL \cup LLL \cup \dots$.

Il problema più interessante rispetto all'approccio riconoscitivo che tratteremo in seguito è la decidibilità di L , ovvero stabilire se esiste un algoritmo che, per qualunque stringa w , è in grado di decidere se $w \in L$ oppure no. Investigheremo inoltre la decidibilità di altri problemi e per alcuni di essi descriveremo gli algoritmi in grado di calcolare la risposta.

Un risultato interessante nella teoria dei linguaggi formali è che l'insieme di tutti i possibili linguaggi definiti su un qualsiasi alfabeto non è ricorsivamente enumerabile.¹ La dimostrazione è un tipico esempio di applicazione del metodo della diagonale, di cui diamo ora prova. Fissato Σ , esiste un criterio per enumerare in una lista $w_1, w_2, \dots, w_i, \dots$ le stringhe appartenenti a Σ^* .² Supponiamo per assurdo che anche tutti i linguaggi su Σ possano essere elencati in una lista $L_1, L_2, \dots, L_i, \dots$. Costruiamo quindi una tabella le cui infinite righe/colonne elencano i linguaggi/stringhe su Σ e tale che ogni cella identificata da una stringa e da un linguaggio è marcata con $\times$ se la stringa appartiene al linguaggio.

¹Un insieme A è ricorsivamente enumerabile se i suoi elementi possono essere elencati (enumerati) tramite un algoritmo. In particolare, se A è infinito, allora esiste una biiezione tra A e l'insieme dei numeri naturali.

²A dispetto dell'intuizionismo, un algoritmo di enumerazione esiste e procede ordinando le stringhe in base alla loro lunghezza e all'ordine lessicografico (predefinito su Σ) dei simboli che le compongono.

	w_1	w_2	w_3	$\dots$
L_1		$\times$	$\times$	$\dots$
L_2		$\times$		$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Tabella 1: esempio per il metodo della diagonale

In Tabella 1 mostriamo un esempio, dove $w_1 \notin L_1$, $w_2 \in L_1$, $w_3 \in L_1$ e così via. Ora definiamo il seguente linguaggio costruito in base alle marcature presenti sulla diagonale della tabella:

$$\bar{L} = \{w_i \mid w_i \notin L_i\}$$

Seguendo l'esempio, abbiamo $w_1 \in \bar{L}$ dato che $w_1 \notin L_1$, ma $w_2 \notin \bar{L}$ dato che $w_2 \in L_2$. Il lettore può convincersi del fatto che $\bar{L}$ non può essere nessuno dei linguaggi presenti nella lista $L_1, L_2, \dots, L_i, \dots$, contraddicendo quindi l'ipotesi che tale lista si possa costruire. Assumiamo infatti che esista j tale che $\bar{L} = L_j$ e chiediamoci se $w_j \in L_j$. Se la risposta è *no*, l'assurdo deriva dal fatto che, per definizione, dovremmo avere $w_j \in \bar{L}$. Se la risposta è *si*, l'assurdo deriva dal fatto che, per definizione, dovremmo avere $w_j \notin \bar{L}$.

Una conseguenza di questo risultato (dimostrabile seguendo lo stesso procedimento) è che esistono una infinità di linguaggi formali non descrivibili tramite alcuna grammatica. È chiara l'analogia con il noto risultato per cui esistono problemi che non sono decidibili, come ad esempio mostrato da Turing [1937] e, indipendentemente e con qualche mese di anticipo, da Church [1936], a fronte della questione posta da Hilbert sull'esistenza di un algoritmo universale per la verifica di qualunque asserzione matematica (Entscheidungsproblem). Church trova un problema indecidibile nel contesto del λ -calcolo, formalismo

da lui proposto, e della classe delle funzioni ricorsive.³ Da parte sua, Turing dimostra l'indecidibilità del cosiddetto halting problem.⁴ A questo scopo, Turing introduce una macchina astratta che ancora oggi rappresenta uno dei modelli più noti per esprimere processi di computazione (e quindi simulare la logica dei calcolatori elettronici). La MdT ha la stessa espressività di sistemi formali (ovvero definiscono la stessa classe di oggetti calcolabili) come il λ -calcolo di Church e la classe delle funzioni ricorsive. Proprio la dimostrazione di equivalenza tra questi due ultimi sistemi, dovuta principalmente a Kleene, ha condotto alla congettura di Church secondo cui non esisterebbero modelli di computazione più espressivi. Quindi i problemi che possono essere risolti algebricamente sono esattamente i problemi che possono essere risolti tramite MdT (cfr. Davis [1985] per una panoramica su questi argomenti).

La MdT è un modello ideale, basato su una struttura di memoria data da un nastro potenzialmente infinito e da un sistema di controllo per l'accesso alle celle del nastro, le quali possono contenere simboli di un dato alfabeto Σ . Il sistema di controllo ha un proprio stato interno e può muoversi lungo il nastro per effettuare operazioni elementari come leggere, scrivere, o cancellare simboli. Quindi, oltre che da Σ , la MdT è specificata da un insieme finito Q di stati che il sistema di controllo può assumere (tra cui si distinguono lo stato iniziale da cui ogni computazione parte e lo stato di terminazione che ne determina la fermata) e da una funzione di transizione P che descrive il comportamento della macchina. Ogni transizione è descritta da cinque elementi: lo stato corrente, il simbo-

³Il processo di ricorsione consente di definire il valore di una funzione in termini di altri valori della stessa funzione.

⁴Il problema della fermata è descritto come segue: dati un programma ed un qualunque input per tale programma, decidere se la sua esecuzione termina oppure no.

lo corrente, il simbolo con cui sostituire quello corrente, il nuovo stato e la direzione di movimento. Ad esempio, $P(q, a) = (b, p, dx)$ stabilisce che se il sistema di controllo è nello stato q ed è posizionato sul simbolo a , allora a viene sostituito con b , il sistema di controllo passa allo stato p e si muove sulla prossima cella a destra nel nastro (assumendo che dx stia per la mossa a destra e sx per la mossa a sinistra).

Per quanto sopra accennato, il modello della MdT rappresenta funzioni calcolabili, ovvero funzioni esprimibili attraverso il comportamento di algoritmi. In particolare, un dato x in ingresso ad una funzione calcolabile f viene espresso da una sequenza finita di simboli presente sul nastro della MdT che computa f , mentre ciò che rimane sul nastro al termine della computazione rappresenta il risultato $f(x)$.⁵

Tra le varie applicazioni della MdT abbiamo anche il problema del riconoscimento di linguaggi formali. Ma prima di caratterizzare i linguaggi riconoscibili tramite MdT, dobbiamo ripercorrere il modo in cui questo modello ha ispirato e influenzato la teoria degli automi.

3. ESPRESSIONI REGOLARI E AUTOMI A STATI FINITI

La più semplice macchina astratta progettata per risolvere il problema del riconoscimento di stringhe è l'automa a stati finiti (ASF) introdotto da Kleene [1951] in un memorandum della U.S. Air Force e pubblicato solo 5 anni dopo (cfr. Kleene [1956]). Il lavoro è ispirato all'analisi logica del comportamento del cervello umano proposta da McCulloch e Pitts [1943], i quali definiscono un modello computazionale a tempo discreto per la rappresentazione astratta di stimoli e reazioni in reti di neuroni e sinapsi. L'obiettivo di Kleene è più

⁵La computazione della MdT potrebbe non terminare per un input x , nel qual caso $f(x)$ non è definito.

generale e va nella direzione di investigare un modello per descrivere il comportamento di sistemi basati su eventi.⁶

Un automa a stati finiti è composto di una memoria finita, espressa dagli stati dell'automa, e di una struttura per la rappresentazione degli eventi che comportano cambiamenti di stato, espressi come transizioni.

Prendiamo come esempio il seguente automa descritto in forma tabellare:

	<i>a</i>	<i>b</i>
$\rightarrow q_0$	q_1	q_0
$* q_1$	q_1	q_1

Le righe riportano gli stati dell'automa, q_0 e q_1 , di cui q_0 , evidenziato dalla freccia, rappresenta lo stato iniziale. Sulle colonne sono riportati i simboli, a e b , che rappresentano i possibili eventi. Per ogni casella individuata da uno stato q_i e da un simbolo s , la tabella riporta lo stato q_j raggiungibile da q_i eseguendo una transizione innescata da s , denotata $q_i \xrightarrow{s} q_j$. In altre parole, se l'automa si trova nello stato q_i ed osserva s allora evolve nello stato q_j . L'automa ha uno stato finale, q_1 , indicato con un asterisco. A partire dallo stato iniziale, la lettura di una sequenza di simboli (da sinistra verso destra) induce un cammino per effetto dell'esecuzione delle transizioni corrispondenti, una per ogni simbolo letto. Se al termine della lettura della sequenza l'automa si trova in uno stato finale, allora la sequenza viene riconosciuta ed accettata. Altrimenti, l'automa la rifiuta. Ad esempio, il cammino $q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1$ è una prova del fatto che l'automa sopra descritto accetta la

⁶Kleene esordisce affermando: «An organism or robot receives certain stimuli and performs certain actions» (cfr. Kleene [1951], p. 1). Successivamente mostra che «a particular example of a finite automaton is a McCulloch and Pitts nerve net» (cfr. Kleene [1951], p. 76).

sequenza ba . È importante osservare che questo automa è *deterministico*: per ogni stato esiste una ed una sola transizione uscente per ciascun simbolo. Questa assunzione vale anche per il modello di Kleene.

In termini matematici, un ASF è una tupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

dove Q è un insieme finito di stati, tra cui si distinguono lo stato iniziale q_0 e l'insieme di stati finali (o di accettazione) $F \subseteq Q$, Σ è l'alfabeto dei simboli leggibili dall'automa, mentre $\delta : (Q \times \Sigma) \rightarrow Q$ è la funzione di transizione che descrive il comportamento dell'automa, ovvero $q_i \xrightarrow{s} q_j$ se $\delta(q_i, s) = q_j$. La funzione δ è totale (ovvero definita su ogni elemento del suo dominio) per esprimere il carattere deterministico dell'automa.

Per definire il linguaggio riconosciuto da un ASF dobbiamo estendere la funzione di transizione introducendone una che formalizzi il concetto di cammino. In particolare, definiamo $\hat{\delta} : (Q \times \Sigma^*) \rightarrow Q$ tale che $\hat{\delta}(q, w)$ restituisce lo stato raggiunto dopo aver letto tutti i simboli in w e assumendo di partire dallo stato q . Formalmente, sfruttiamo il principio di induzione per darne la definizione:

$$\begin{aligned}\hat{\delta}(q, \epsilon) &= q \\ \hat{\delta}(q, xa) &= \delta(\hat{\delta}(q, x), a)\end{aligned}$$

dove $a \in \Sigma$ e $x \in \Sigma^*$. Il passo base ci dice che quando non ha nulla da leggere, l'automa rimane nello stato corrente. Il passo induttivo stabilisce che lo stato raggiunto leggendo la sequenza xa si determina applicando δ ad a e allo stato che si raggiunge leggendo la

sequenza x . Quindi, $\hat{\delta}$ è una funzione ricorsiva, ovvero per calcolarne il valore per una stringa di lunghezza n occorre applicare la stessa funzione ad una stringa di lunghezza $n - 1$ (e questo fino a giungere al caso base, che ci permette di ricostruire il risultato).

Il linguaggio $L(M)$ accettato da un automa M è l'insieme di stringhe che fanno evolvere l'automata in uno stato finale a partire dallo stato iniziale:

$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}$$

Kleene [1951] offre una caratterizzazione alternativa per esprimere le sequenze di eventi riconosciuti da ASF. Lo fa per mezzo di espressioni sintattiche cosiddette regolari, costruite composizionalmente a partire dagli elementi atomici (ovvero i simboli che rappresentano gli eventi) e usando tre soli connettivi: unione (operatore $+$), concatenazione (giustapposizione di simboli), e stella di Kleene (operatore $*$), la quale consiste di zero o più occorrenze consecutive del termine cui viene applicata.⁷ Ogni espressione regolare rappresenta un insieme di stringhe. Ad esempio, $a + b$ denota l'insieme $\{a, b\}$, a^* rappresenta l'insieme infinito $\{\epsilon, a, aa, aaa, \dots\}$, mentre la semantica dell'espressione regolare $(a + b)^*$ è data dall'insieme infinito di stringhe $\{a\} \cup \{b, bb, bbb, \dots\}$. Va notato che in realtà Kleene non definisce la stella come operatore unario, bensì come operatore binario a^*b intendendo in questo modo qualunque sequenza:

$$\overbrace{a \dots a}^n b$$

⁷Una caratterizzazione alternativa delle espressioni regolari esiste nell'ambito delle teorie algebrico-combinatoriali (cfr. Chomsky e Schützenberger [1963] e Schützenberger [1965]).

per $n \geq 0$, ovvero zero o più occorrenze di a seguite infine da b . Il motivo è che nel modello presentato da Kleene [1951] gli eventi sono temporalmente definiti e di durata fissa,⁸ compatibilmente con le reti di McCulloch e Pitts, per cui sequenze di durata nulla (che a^* può creare) non sono ammissibili.

Limitandoci ad una trattazione algebrica delle espressioni regolari che esula da interpretazioni dei simboli (e quindi da aspetti di natura temporale), diamone ora una definizione formale che tenga conto anche di insieme vuoto e stringa vuota. Tale definizione è parametrica rispetto ad un fissato alfabeto Σ e si basa sul principio di induzione. Le espressioni regolari di base sono $\emptyset$, ϵ , e a (per ogni $a \in \Sigma$), che, rispettivamente, rappresentano l'insieme vuoto, l'insieme $\{\epsilon\}$ contenente la stringa vuota, e l'insieme $\{a\}$ contenente la stringa a . Se E ed F sono espressioni regolari che rappresentano gli insiemi di stringhe $\mathcal{E}$ e $\mathcal{F}$, allora $E + F$, EF , E^* , e (E) sono espressioni regolari che, rispettivamente, rappresentano $\mathcal{E} \cup \mathcal{F}$, $\mathcal{EF}$, $\mathcal{E}^*$, mentre le parentesi non alterano la semantica di E ma servono solo a specificare l'ordine di precedenza degli operatori.

Il teorema fondamentale dimostrato da Kleene concerne l'equivalenza tra il modello degli ASF e il modello delle espressioni regolari. In altri termini, la classe dei linguaggi riconosciuti da automi a stati finiti coincide con la classe dei linguaggi generati da espressioni regolari. Tali linguaggi vengono quindi chiamati regolari. La dimostrazione del teorema è costruttiva. In particolare, l'implicazione da espressione regolare ad ASF sfrutta la definizione ricorsiva della prima ed è basata su una semplice induzione sulla struttura dell'espressione stessa. Intuitivamente, l'idea consiste nel definire gli automi per le

⁸«We shall first restrict ourselves to events which refer to a fixed period of time» (cfr. Kleene [1951], p. 10).

espressioni regolari atomiche e costruire quindi automi più complessi tramite i tre operatori di composizione. Il primo algoritmo efficiente per tale trasformazione viene proposto da McNaughton e Yamada [1960]. L'implicazione opposta è più complessa e si basa sulla traduzione di tutti i possibili cammini di accettazione in espressioni regolari. In questo caso la dimostrazione è per induzione sul numero di diversi stati visitati da tali cammini. L'espressione regolare che descrive il linguaggio riconosciuto dall'automata del nostro esempio è $b^*a(a+b)^*$, ovvero l'insieme di stringhe che contengono almeno una occorrenza del simbolo a . Si noti infatti che tali stringhe sono composte di una sequenza eventualmente vuota di b (espressa nell'automata dalla transizione $q_0 \xrightarrow{b} q_0$), seguita dal simbolo a (espressa nell'automata dalla transizione $q_0 \xrightarrow{a} q_1$) e quindi da una qualunque sequenza generata dall'espressione regolare $(a+b)^*$ (espressa nell'automata dalle transizioni uscenti da q_1), la quale rappresenta l'insieme di stringhe $\{a, b\}^*$, cioè la chiusura di Kleene dell'alfabeto $\{a, b\}$.

Tornando a Kleene, è importante chiedersi quale espressività caratterizzi i suoi modelli. Kleene stesso anticipa il lettore svelando il motivo dietro la scelta dei tre operatori di base per le espressioni regolari. E la risposta che fornisce è la semplicità, che è alla base del risultato di decidibilità del suo teorema fondamentale. Sebbene dichiari di non aver investigato le implicazioni di tale scelta, Kleene prevede che la teoria introdotta «will prove handy in describing events» (cfr. Kleene [1951], p. 71), a giustificazione delle restrizioni imposte, ad esempio rispetto alla MdT, di cui è perfettamente consapevole. Non a caso il memorandum di Kleene termina con un richiamo a tale confronto:

A machine of Turing (1937) which is supplied with an unlimited amount of tape, is not a finite automaton in our present sense, since, although in its operation only a

finite number of squares of tape are printed upon at any time, there is no preassigned bound to this number.⁹

Kleene descrive quindi la MdT come una estensione degli ASF in grado di gestire una memoria ausiliaria, il nastro potenzialmente infinito. Una valutazione più formale dell'espressività degli ASF verrà data nella prossima sezione.

A ulteriore prova della semplicità di trattamento della teoria di Kleene, citiamo il fatto che i linguaggi regolari sono chiusi non solo per unione, concatenazione e chiusura di Kleene, ma anche per intersezione, complemento e differenza (l'applicazione di tali operatori a linguaggi regolari restituisce un linguaggio regolare). Inoltre, per i linguaggi regolari sono decidibili i più comuni problemi, tra cui appartenenza ($w \in L$), test del vuoto ($L = \emptyset$), inclusione ($L_1 \subseteq L_2$), finitezza di L , ed equivalenza ($L_1 = L_2$).

È interessante notare che le dimostrazioni più semplici di molte di tali proprietà si ottengono in realtà nell'ambito degli ASF nondeterministici. L'idea del nondeterminismo in questo contesto è che in qualche stato l'automa potrebbe, in risposta ad un certo simbolo, scegliere tra diversi stati verso cui muovere o, in alternativa, bloccarsi e quindi rifiutare la stringa o, infine, evolvere tramite una mossa ϵ senza dover leggere alcun simbolo dalla stringa. Formalmente, la funzione di transizione diventa $\delta : (Q \times (\Sigma \cup \{\epsilon\})) \rightarrow \mathcal{P}(Q)$ dove $\mathcal{P}(Q)$ denota l'insieme delle parti di Q , ovvero tutti i possibili sottinsiemi di Q , insieme vuoto incluso. Ad esempio, se $\delta(q, s) = \{p_1, p_2\}$ allora l'automa, se in q e leggendo s , può evolvere in p_1 oppure in p_2 . Un effetto immediato del nondeterminismo è che ad una sequenza di simboli potrebbero essere associati diversi possibili cammini. È però

⁹Cfr. Kleene [1951], p. 86.

sufficiente che uno di questi conduca ad uno stato finale dopo aver letto l'intera stringa per poterne determinare l'accettazione.

Riprendendo il discorso del legame tra espressioni regolari e ASF, è immediato osservare che l'automa nondeterministico che riconosce l'espressione regolare atomica a si definisce come $(\{q_0, q_1\}, \{a\}, \delta, q_0, \{q_1\})$, dove δ descrive l'unica transizione $q_0 \xrightarrow{a} q_1$. D'altra parte, l'automa nondeterministico M che riconosce l'espressione regolare complessa $E_1 + E_2$ si ottiene per induzione combinando gli ASF M_1 e M_2 associati ad E_1 ed E_2 , rispettivamente. In particolare, lo stato iniziale di M è connesso agli stati iniziali di M_1 e M_2 per mezzo di due transizioni ϵ , le quali modellano la possibilità di riconoscere una stringa tramite M_1 o tramite M_2 , a seconda di come viene risolta la scelta nondeterministica.

L'interpretazione del nondeterminismo in questo contesto è che un oracolo è in grado, ad ogni passo, di risolvere eventuali scelte in modo da indovinare il cammino di accettazione, se esiste. Più pragmaticamente, un informatico si affiderebbe a diversi processori, uno per ogni possibile cammino, che in parallelo esplorano le diverse strategie. Un informatico con meno risorse userebbe un unico calcolatore e la tecnica del *backtracking*. L'idea è che non ha importanza quale cammino si segua, in quanto basta tenere traccia di ogni punto di scelta, in modo tale da poter ritornare all'ultima scelta effettuata qualora la strada intrapresa non sia di successo. Qualunque sia l'interpretazione che diamo del nondeterminismo e di come affrontarlo, l'idea che se ne trae è che consenta una maggior espressività al prezzo di un maggior dispendio di risorse. Senza addentrarci nello studio di questi problemi, che sono alla base della teoria della complessità computazionale (cfr. Hopcroft,

Motwani, e Ullman [2007]), limitiamoci ad osservare cosa comporta l'introduzione del nondeterminismo nel contesto della teoria degli automi.

Gli automi a stati finiti nondeterministici sono introdotti da Rabin e Scott [1959], nel cui lavoro si trova la dimostrazione della loro equivalenza in termini di espressività con gli ASF deterministici di Kleene. In altre parole, esiste un procedimento per trasformare un ASF nondeterministico in uno deterministico che riconosce lo stesso linguaggio regolare. Rispetto alla questione delle teorie decidibili, il modello degli ASF ha dato un contributo importante nell'ambito della logica, primo in ordine di tempo quello di Richard Büchi. A margine della fondamentale prova di incompletezza di Gödel [1931], ovvero la teoria logica del primo ordine per gli interi con gli operatori di somma e moltiplicazione non è decidibile,¹⁰ il risultato positivo più rilevante era dovuto a Pressburger [1929], ovvero la stessa teoria limitata all'operatore $+$ è decidibile. Attraverso una caratterizzazione basata su ASF, Büchi [1960] dimostra la decidibilità della teoria monadica del secondo ordine degli interi con successore. L'idea di Büchi consiste nel rappresentare insiemi di interi come sequenze binarie potenzialmente infinite (la sequenza 010101... corrisponde all'insieme dei numeri dispari), per poi dimostrare che le sequenze generate dalla teoria di cui sopra sono esattamente espressioni regolari, da cui quindi segue il risultato di decidibilità. In seguito, ASF estesi per il riconoscimento di stringhe infinite prendono il nome di automi di Büchi. Altre caratterizzazioni di teorie logiche in termini di automi a

¹⁰Per teoria logica intendiamo un sistema formale di asserzioni, detti assiomi, utile a dedurre asserzioni, detti teoremi, in termini di conseguenze logiche degli assiomi. Esempi sono i sistemi di Hilbert, l'aritmetica di Peano e la teoria degli insiemi di Zermelo-Fraenkel. Una teoria logica si dice decidibile se esiste una procedura automatica finita per verificare se una asserzione rappresenta un teorema di tale teoria. Ad esempio, la logica proposizionale lo è (basta costruire la tavola di verità della formula da verificare), mentre logica del primo ordine, aritmetica di Peano e teoria degli insiemi ZF non lo sono.

stati finiti hanno permesso di esprimere numerosi risultati di decidibilità, come nel caso delle logiche modali e temporali e le relative applicazioni nel campo della verifica del software.¹¹

4. LINGUAGGI LIBERI E AUTOMI A PILA

Per mettere in evidenza le limitazioni degli ASF e giustificare un salto di espressività, diventa utile approfondire il tema dell'approccio generativo allo studio dei linguaggi formali.

Una grammatica è un sistema di riscrittura basato su regole (che nel seguito chiameremo *produzioni*) per la generazione di stringhe a partire da una certa parola o simbolo di base. In altri termini, se intendiamo tale base iniziale come l'assioma della grammatica e interpretiamo le produzioni come regole di inferenza, possiamo assimilare una grammatica ad un sistema deduttivo. Quali stringhe considerare tra quelle deducibili è una questione che si risolve partizionando l'alfabeto dei simboli di riferimento in due insiemi: i simboli *terminali* sono quelli che compongono le stringhe del linguaggio generato dalla grammatica, i simboli nonterminali (detti anche *variabili*) sono solo usati dalle produzioni durante il procedimento di derivazione delle stringhe. Di solito, tra le variabili se ne individua una che funge da simbolo iniziale di base.

I sistemi di riscrittura non rappresentano una novità apparsa con le grammatiche formali, ma risalgono ai sistemi Semi-Thue definiti dal matematico norvegese Thue [1914] con l'obiettivo di investigare metodi di verifica automatica di deduzioni logiche nel contesto

¹¹Un mirabile esempio è il lavoro fondazionale di Vardi e Wolper [1986].

di un linguaggio formale.¹²

Formalmente, una grammatica è una tupla:

$$G = \{V, T, S, P\}$$

dove V è l'insieme delle variabili, di cui S rappresenta quella iniziale, T è l'insieme dei terminali, che quindi rappresenta l'alfabeto per il linguaggio generato dalla grammatica, P è un insieme di produzioni del tipo $\alpha \rightarrow \beta$, dove α (non vuota) e β (possibilmente vuota) sono due sequenze di simboli terminali e/o nonterminali. Intuitivamente, una produzione $\alpha \rightarrow \beta$ stabilisce che α può essere sostituita sintatticamente con β in qualunque contesto. Formalmente, la grammatica G induce una relazione di derivazione $\Rightarrow_G$ tale che la stringa $w = \gamma\beta\mu$ deriva dalla stringa $v = \gamma\alpha\mu$, scritto $v \Rightarrow_G w$, se esiste la produzione $\alpha \rightarrow \beta$ in G . In altri termini, le produzioni rappresentano regole di riscrittura sintattica, il cui unico vincolo ($\alpha \neq \epsilon$) stabilisce che dal nulla non si genera. L'applicazione della relazione di derivazione $\Rightarrow_G$ si può reiterare alla maniera della stella di Kleene. Intuitivamente, $v \Rightarrow_G^* w$ se w è derivabile da v in zero o più passi di derivazione, in ognuno dei quali si applica una produzione che determina una sostituzione sintattica nella stringa corrente. Il linguaggio $L(G)$ generato da una grammatica G si definisce come segue:

$$L(G) = \{w \in T^* \mid S \Rightarrow_G^* w\}$$

Prendiamo ad esempio una grammatica con tre variabili – S (variabile iniziale), A e B –

¹²Una interessante analisi e traduzione dei lavori di Thue è di Berstel [1995].

due terminali – 0 e 1 – e le seguenti produzioni: $S \rightarrow A$, $A \rightarrow 0A$, $A \rightarrow B$, $B \rightarrow 1B$, $B \rightarrow \epsilon$. La derivazione:

$$S \rightarrow A \rightarrow 0A \rightarrow 00A \rightarrow 00B \rightarrow 001B \rightarrow 001$$

dimostra come 001 sia una stringa generata dalla grammatica. L'insieme di tutte le possibili stringhe di terminali derivate a partire dal simbolo S determina il linguaggio generato dalla grammatica. Il lettore dovrebbe convincersi del fatto che, nel nostro esempio, tale linguaggio sia l'insieme di tutte le stringhe binarie le cui occorrenze di 0 precedono tutte le occorrenze di 1.¹³

Nel lavoro fondazionale di Chomsky [1957] viene proposta una gerarchia di classi di grammatiche che si differenziano tra loro rispetto alle restrizioni che vengono applicate alla forma delle produzioni. Tali restrizioni determinano l'espressività delle diverse classi.¹⁴ In ordine decrescente di espressività, Chomsky battezza quattro classi da *Type-0* a *Type-3* (cfr. Jiang, Li, Ravikumar, e Regan [2010] per una breve introduzione). Ad esempio, se la definizione formale vista poc'anzi corrisponde alle grammatiche *Type-0* (cosiddette generali), le grammatiche *Type-2*, chiamate libere, sono vincolate da produzioni del tipo $A \rightarrow \beta$, dove A è un simbolo nonterminale. In seguito, per caratterizzare ciascuna classe, sono state studiate semplificazioni per le produzioni che le riconducessero a qualche forma normale senza alterarne l'espressività. Ad esempio, tra le più note per le grammatiche libere, citiamo la forma normale di Greibach [1981], per la quale tutte

¹³Si noti come lo stesso linguaggio sia generato dalla espressione regolare 0^*1^* .

¹⁴Una classe di grammatiche A è più espressiva di un'altra classe B se l'insieme di linguaggi generati in A contiene propriamente l'insieme di linguaggi generati in B .

le produzioni sono riconducibili alla forma $A \rightarrow aw$, dove a è un terminale e w è una sequenza potenzialmente vuota di variabili. Se si pone il limite che w contenga al più una variabile otteniamo la forma che vincola le grammatiche *Type-3*.

La motivazione originale per cui Chomsky introduce la sua gerarchia è che diversi aspetti del linguaggio naturale richiedono classi di grammatiche diverse per espressività. Ad esempio, la morfologia della lingua inglese (e quindi il vocabolario dei termini che si usano per costruire frasi) è modellabile tramite una grammatica meno espressiva di quanto invece necessario per caratterizzare la sintassi della lingua inglese (e quindi costruire frasi).¹⁵ Per dare un esempio del legame tra grammatiche libere e le tipiche regole sintattiche del linguaggio naturale, consideriamo il seguente frammento:

Frase $\rightarrow$ Soggetto Verbo Oggetto
 Soggetto $\rightarrow$ *Alex* | *Mary*
 Verbo $\rightarrow$ *mangia* | *beve*
 Oggetto $\rightarrow$ *banane* | *birra*

dove i termini in corsivo rappresentano i terminali della grammatica, mentre gli altri termini rappresentano le variabili, di cui Frase è quella iniziale (la notazione $A \rightarrow \alpha_1 \mid \alpha_2$ è una forma contratta per $A \rightarrow \alpha_1$ e $A \rightarrow \alpha_2$). Quindi, in quattro passi di derivazione, dalla variabile iniziale Frase è possibile dedurre la sequenza di terminali *Alex beve birra*. Da notare che la grammatica descrive la sintassi del linguaggio, astruendo completamente dalla semantica.

¹⁵Nel primo caso basta una grammatica *Type-3*, mentre nel secondo occorre una grammatica *Type-2*.

Quanto ipotizzato a partire dalle idee di Chomsky, come l'asserzione secondo cui il cervello umano sarebbe naturalmente "programmato" per utilizzare un insieme di base di regole di riscrittura (la cosiddetta *grammatica universale*), va oltre la teoria dei linguaggi formali. Quel che è certo invece è lo stretto legame tra la teoria di Chomsky e, ad esempio, lo sviluppo dei linguaggi di programmazione.

Solo due anni dopo l'opera del 1957 di Chomsky, Backus [1959] presenta la grammatica (libera) per un linguaggio di programmazione, con l'obiettivo di formalizzarne la struttura sintattica e favorire quindi un procedimento automatico di compilazione. Un compilatore è un programma traduttore che trasforma un programma sorgente scritto in un linguaggio ad alto livello di astrazione (come Pascal, C, e Java) in un programma oggetto scritto in un linguaggio a più basso livello (come il linguaggio macchina). L'idea di un procedimento automatico di traduzione da un linguaggio "vicino" a quello naturale e comprensibile dall'uomo ad un linguaggio più simile alla logica dei circuiti digitali è stata fondamentale per lo sviluppo accelerato delle tecnologie informatiche. Nelle fasi iniziali del processo di traduzione, il compilatore effettua sul programma sorgente analisi di tipo lessicale, per la verifica dei termini usati, e di tipo sintattico, per la verifica della struttura delle istruzioni. Analogamente alle tesi di Chomsky su morfologia e sintassi del linguaggio naturale, gli strumenti richiesti per tali analisi sono diversi per espressività e tecniche usate. Parafrasando l'esempio di grammatica libera per un linguaggio naturale visto in precedenza, un frammento per un linguaggio di programmazione che descrive la sintassi di istruzioni condizionali e cicli può essere come segue:

Statement $\rightarrow$ *if* Boolean_condition Statement *else* Statement |
while Boolean_condition Statement

Boolean_condition $\rightarrow$ *true* | *false* | ...

Il lavoro di Backus [1959] mette in evidenza le implicazioni della formalizzazione dei linguaggi cosiddetti liberi (ovvero generati da grammatiche libere).¹⁶ Diventa quindi importante stabilire quale classe di automi sia necessaria per applicare l'approccio riconoscitivo allo studio di tali linguaggi. Purtroppo, la classe dei linguaggi regolari risulta essere propriamente inclusa nella classe dei linguaggi liberi, che quindi non sono rappresentabili tramite ASF. In questa lacuna, il limite degli ASF deriva dalla memoria finita, in quanto il numero di stati è fissato a priori e non dipende dinamicamente dalla lunghezza della stringa da riconoscere. Ad esempio, si consideri la grammatica libera $G_1 = \{\{S\}, \{a, b\}, S, \{S \rightarrow aSb \mid ab\}\}$, che genera $L(G_1) = \{a^n b^n \mid n > 0\}$, ovvero sequenze del simbolo a concatenate a sequenze di pari lunghezza del simbolo b . Un ASF riconoscitore dovrebbe avere una memoria in grado di "ricordare" il numero di occorrenze di a che vengono lette. Tuttavia, la memoria di un ASF sono gli stati, e per ricordarsi di aver letto a^n sarebbero necessari n stati, da cui segue l'assurdo dato che n è un intero grande a piacere mentre lo spazio degli stati dell'automa è limitato e fissato a priori. Quindi sorge spontanea la questione di capire quale sia effettivamente l'espressività degli ASF di Kleene. La risposta la troviamo in Chomsky e Miller [1958], dove si dimostra che la classe dei linguaggi riconosciuti da ASF coincide con la classe di linguaggi generati da grammatiche *Type-3* di Chomsky. Quindi, per trattare i linguaggi liberi dobbiamo

¹⁶Idee fondazionali erano già presenti in Turing [1937] e Post [1936].

introdurre una nuova classe di automi.

Gli automi a pila sono automi a stati finiti arricchiti con una memoria ausiliaria potenzialmente infinita, chiamata *stack* o *pila*, che può essere sia letta che scritta. Il modello viene definito da Ginsburg, Greibach, e Harrison [1967] come strumento formale per il controllo sintattico durante la compilazione di un programma. La caratteristica peculiare dello stack è quella di essere accessibile solamente in modalità *last-in-first-out*. Se paragoniamo lo stack ad una pila di piatti da lavare, tale politica di accesso coincide con la regola che userebbe un lavapiatti che ha cura per il servizio, ovvero prendendo o aggiungendo sempre il piatto in cima alla pila. In virtù della presenza dello stack, ogni transizione diventa funzione dello stato corrente, del prossimo simbolo della stringa da leggere e del simbolo presente in cima alla pila, mentre per effetto della transizione non solo l'automa cambia stato, ma il simbolo in cima alla pila viene sostituito con una nuova sequenza di simboli. Il criterio di riconoscimento di una stringa può essere determinato dal raggiungimento di uno stato di accettazione, come nel caso degli ASF, oppure *per pila vuota*, ovvero al termine della lettura della stringa la pila deve essere vuota. I due criteri risultano essere equivalenti. Non sono invece ugualmente espressivi automi a pila deterministici e nondeterministici, essendo i secondi strettamente più espressivi dei primi. Ad esempio, il nondeterminismo risulta indispensabile per riconoscere tramite automa a pila il linguaggio $L(G_1)$ precedentemente descritto. Vediamo come agirebbe tale automa nel riconoscere una generica stringa $w \in L(G_1)$. Nello stato iniziale, ad ogni mossa l'automa è disposto a leggere un simbolo a , copiarlo nello stack e rimanere in tale stato. In maniera nondeterministica, è disposto anche a transitare in un nuovo stato, dove l'automa

è disposto a leggere un simbolo b , prelevare un simbolo a dallo stack e rimanere in tale stato. Se al termine della lettura della stringa lo stack è vuoto, significa che sono state lette tante a quante b e la stringa viene accettata. Ovviamente, per come abbiamo definito gli automi nondeterministici, sappiamo che, tra le tante possibili, esiste una strategia che permette all'automata di "indovinare" il momento in cui transitare dal primo al secondo stato e quindi riconoscere la stringa.

Formalmente, un automa a pila nondeterministico è una tupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

dove, rispetto agli ingredienti già noti per gli ASF, abbiamo l'alfabeto Γ dei simboli per lo stack, tra cui si distingue il simbolo iniziale Z_0 , che rappresenta il contenuto dello stack all'inizio della computazione.

La funzione di transizione è $\delta : (Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma) \rightarrow \mathcal{P}(Q \times \Gamma^*)$. Ad esempio, se $(p, \gamma) \in \delta(q, a, z)$, dalla configurazione in cui q è lo stato corrente, a il prossimo simbolo da leggere dalla stringa e z il simbolo in cima alla pila, l'automata può evolvere transitando nello stato p , leggendo a e scrivendo sulla pila la sequenza γ in sostituzione di z .

La classe dei linguaggi riconosciuti da automi a pila nondeterministici coincide con la classe dei linguaggi generati da grammatiche *Type-2* di Chomsky. A questo proposito, proponiamo al lettore interessato la traduzione più semplice, quella da una grammatica libera $G = (V, T, S, P)$ all'automata che riconosce $L(G)$ per pila vuota, definito come $M = (\{q\}, T, V \cup T, \delta, q, S)$. Lo stato è unico, con ruolo del tutto marginale, l'alfabeto è l'insieme dei terminali T della grammatica, la pila può ospitare variabili e terminali

(inizialmente contiene la variabile S da cui ogni derivazione parte), mentre non serve fissare un insieme di stati di accettazione. La funzione di transizione δ simula i passi di derivazione che dimostrano quali stringhe appartengono al linguaggio, utilizzando la pila come supporto di memoria temporaneo dove parcheggiare i risultati intermedi della derivazione. In particolare, per ogni variabile $A \in V$:

$$\delta(q, \epsilon, A) = \{(q, \beta) \mid A \rightarrow \beta \in P\}$$

ovvero se in testa alla pila c'è una variabile A allora la possiamo sostituire con β , simulando quindi l'applicazione della produzione $A \rightarrow \beta$. In tal caso nessun simbolo viene letto dalla stringa da riconoscere. Per ogni terminale $a \in T$:

$$\delta(q, a, a) = \{(q, \epsilon)\}$$

ovvero se in testa alla pila c'è un terminale a (presente in quanto generato durante il procedimento di derivazione), lo confrontiamo con il prossimo simbolo da leggere nella stringa: se sono uguali il procedimento continua consumandoli entrambi, altrimenti la stringa viene rifiutata. Se il procedimento termina avendo letto tutta la stringa e svuotato la pila, allora il riconoscimento avviene con successo. Quindi, tutta l'espressività necessaria all'automa risiede nello stack. Questo schema di traduzione rappresenta il kernel di molti compilatori per moderni linguaggi di programmazione, dove l'analisi sintattica delle istruzioni rispetto alla grammatica del linguaggio avviene per opera di un modulo, chiamato *parser*, che implementa l'automa a pila riconoscitore del linguaggio.

Torniamo ora ai linguaggi liberi e alle loro complicazioni rispetto alle espressioni regolari. I linguaggi liberi non sono chiusi per intersezione e complemento, mentre lo sono per i tre operatori delle espressioni regolari. I problemi decidibili sono appartenenza, test del vuoto, finitezza ed infine equivalenza (limitatamente a linguaggi liberi riconosciuti da automi a pila deterministici), quest'ultimo dimostrato da Sénizergues [1997]. I problemi di indecidibilità sono molto più numerosi, tra cui citiamo equivalenza (per linguaggi liberi generici), test di intersezione vuota ($L_1 \cap L_2 = \emptyset$) e ambiguità. Il problema dell'ambiguità è particolarmente rilevante. Intuitivamente, una grammatica si dice ambigua quando esistono diverse deduzioni della stessa stringa, mentre un linguaggio si dice intrinsecamente ambiguo se ogni grammatica che lo genera è ambigua. È chiaro quindi che l'ambiguità causa problemi laddove si vuole automatizzare il procedimento di riconoscimento, in quanto deduzioni distinte potrebbero dar luogo a chiavi di lettura completamente diverse di una stringa, con conseguenze negative sulla sua interpretazione semantica. Sebbene scoraggiante da un punto di vista teorico, l'indecidibilità della ambiguità si aggira grazie a tecniche note per la definizione di produzioni la cui forma scongiura il pericolo. Ad esempio, le grammatiche libere sottostanti i moderni linguaggi di programmazione non sono ambigue e i relativi parser sono deterministici. Per completezza, va anche sottolineato che non esistono linguaggi regolari intrinsecamente ambigui.

La dimostrazione dei risultati negativi sopra citati si ottiene per riduzione da un noto problema indecidibile della teoria dei linguaggi formali, detto della corrispondenza di Post (cfr. Post [1946]), la cui definizione spicca per semplicità. Sono date due liste di stringhe $U = \alpha_1, \alpha_2, \dots, \alpha_n$ e $V = \beta_1, \beta_2, \dots, \beta_n$ definite su uno stesso alfabeto di

almeno due simboli. Il problema consiste nel decidere se esiste una sequenza di indici $i_1, \dots, i_k$ tale che $\alpha_{i_1} \cdots \alpha_{i_k} = \beta_{i_1} \cdots \beta_{i_k}$, nel qual caso si dice che $(U; V)$ è una istanza con soluzione. Ad esempio, l'istanza $(a, ba, aab; aa, b, ab)$ ha una sua possibile soluzione nella parola $baaaaab$, ottenuta, sia per U che per V , componendone le stringhe secondo la sequenza di indici 2, 1, 1, 3.

Mostriamo ora come questo problema indecidibile si possa ridurre al test di intersezione vuota tra linguaggi liberi. Date $U = \alpha_1, \alpha_2, \dots, \alpha_n$ e $V = \beta_1, \beta_2, \dots, \beta_n$ definite sull'alfabeto $\{a, b\}$, costruiamo due grammatiche libere sull'alfabeto di terminali $\{a, b, 1, \dots, n\}$, G_U con produzioni:

$$A \rightarrow \alpha_1 A 1 \mid \dots \mid \alpha_n A n \mid \alpha_1 1 \mid \dots \mid \alpha_n n$$

e G_V con produzioni:

$$B \rightarrow \beta_1 B 1 \mid \dots \mid \beta_n B n \mid \beta_1 1 \mid \dots \mid \beta_n n$$

Rispetto al nostro esempio, si noti che la derivazione:

$$A \rightarrow baA2 \rightarrow baaA12 \rightarrow baaaA112 \rightarrow baaaaab3112$$

descrive la sequenza di stringhe di U da considerare per ottenere la parola $baaaaab$. Così come $baaaaab3112 \in L(G_U)$, è altrettanto dimostrabile che $baaaaab3112 \in L(G_V)$. Più in generale, se $L(G_U) \cap L(G_V) \neq \emptyset$, allora abbiamo una soluzione per l'istanza $(U; V)$,

ottenendo di fatto la riduzione da cui segue il risultato di indecidibilità.¹⁷

Chiudiamo il cerchio aperto con i sistemi di riscrittura di Thue osservando che l'approccio usato in Post [1946], dietro suggerimento di Church, viene successivamente usato in Post [1947] per stabilire l'indecidibilità di uno dei problemi fondamentali posti in Thue [1914], ovvero se tramite il suo sistema di riscrittura è possibile trasformare una certa stringa in un'altra data stringa. Il lavoro ha inizio così:

Alonzo Church suggested to the writer that a certain problem of Thue [1914] might be proved unsolvable by the methods of Post [1946]. We proceed to prove the problem recursively unsolvable, that is, unsolvable in the sense of Church [1936], but by a method meeting the special needs of the problem.¹⁸

5. VERSO I CONFINI DEL CALCOLABILE

Se modifichiamo gli automi a pila consentendo l'accesso a qualunque posizione dello stack, otteniamo una estensione nota come automa limitato linearmente. Più precisamente, la struttura di memoria ausiliaria diventa un nastro finito. Inizialmente il nastro contiene la stringa da riconoscere, delimitata sia a sinistra che a destra da due caratteri speciali, diversi tra loro, usati per riconoscere i "confini" del nastro. L'automa è posizionato sul primo simbolo della stringa e può leggere o scrivere sul nastro muovendosi in qualunque direzione, contrariamente agli ASF e a quelli a pila, che devono scandire la stringa da riconoscere da sinistra verso destra.

Storicamente, gli automi limitati linearmente non vengono proposti per estendere gli automi a pila, bensì per vincolare i gradi di libertà della MdT. Myhill [1960] osserva infatti che la MdT rappresenta un modello di calcolo puramente teorico e decide quindi di sviluppar-

¹⁷Se per assurdo $L(G_U) \cap L(G_V) \neq \emptyset$ fosse decidibile, allora per la riduzione appena vista lo sarebbe anche il problema della corrispondenza di Post.

¹⁸Cfr. Post [1947], p. 1.

ne una variante per calcolatori reali. Per questo scopo, è sufficiente limitare la dimensione del nastro a quella dell'input. L'automa definito da Myhill è deterministico e non viene messo in relazione con la teoria dei linguaggi formali. Tre anni più tardi, Landweber affronta il problema e dimostra che gli automi di Myhill riconoscono linguaggi dipendenti dal contesto, ovvero generati da grammatiche *Type-1* di Chomsky (cfr. Landweber [1963]). Poco più tardi, Kuroda [1964] introduce gli automi limitati linearmente nondeterministici e, seguendo la dimostrazione di Landweber, realizza che essi coincidono per espressività con le grammatiche *Type-1* di Chomsky. Se gli automi nondeterministici di Kuroda siano equivalenti a quelli deterministici di Myhill è tuttora un problema aperto, così come non è noto se esistano linguaggi dipendenti dal contesto intrinsecamente ambigui.

Formalmente, un automa limitato linearmente nondeterministico è una tupla:

$$M = (Q, \Sigma, B, \delta, q_0, X_l, X_r, F)$$

dove, agli ingredienti già noti, si aggiungono l'alfabeto B dei simboli su nastro (si noti che $\Sigma \subseteq B$) e i simboli speciali X_l e X_r usati per delimitare i confini sinistro e destro del nastro, rispettivamente. La funzione di transizione $\delta : (Q \times B) \rightarrow \mathcal{P}(Q \times B \times \{dx, sx\})$ determina, fissati stato e simbolo correnti, come questi cambiano e in quale direzione ci si sposta lungo il nastro. Per rispettare le limitazioni di movimento imposte dalla finitezza del nastro, se $(p, a, d) \in \delta(q, X_L)$ allora $a = X_L$ e $d = dx$, ovvero il delimitatore sinistro non può essere modificato e l'automa deve muoversi verso destra. Analogamente, se $(p, a, d) \in \delta(q, X_R)$ allora $a = X_R$ e $d = sx$. Il criterio di riconoscimento è stabilito dal

raggiungimento di una configurazione in cui l'automa si trova in uno stato finale.

I linguaggi dipendenti dal contesto sono chiusi per unione, concatenazione, chiusura di Kleene e, a differenza dei linguaggi liberi, per intersezione e complemento, quest'ultimo uno dei risultati della matematica discreta più importanti degli anni '80. Curiosamente, sebbene rimasto un problema aperto a lungo, è stato risolto nel 1988 in maniera indipendente e contemporanea da Immerman [1988] e da Szelepcsényi [1988]. L'unico problema decidibile rilevante è quello dell'appartenenza, risultato che, come vedremo, rende i linguaggi dipendenti dal contesto la più ampia classe di linguaggi decidibili.

Infine, vogliamo tornare al quesito che ci eravamo posti su quale classe di linguaggi formali fosse riconosciuta dal modello della MdT. Per trattare la MdT come un automa riconoscitore di stringhe, è sufficiente considerare la definizione formale di automa limitato linearmente e rilassare la condizione sulla finitezza del nastro.

Gli automi di Turing deterministici e nondeterministici hanno la stessa espressività e riconoscono una classe chiusa per unione, concatenazione, chiusura di Kleene, intersezione e priva di linguaggi intrinsecamente ambigui. Tale classe coincide con l'insieme dei linguaggi ricorsivamente enumerabili, ovvero la più ampia classe di linguaggi descrivibile tramite grammatiche, quelle *Type-0* secondo la gerarchia di Chomsky. Questa equivalenza fissa definitivamente il legame tra grammatiche, automi e linguaggi formali. Ovviamente, i limiti noti per la calcolabilità si propagano anche al problema del riconoscimento di linguaggi. Ad esempio, i linguaggi ricorsivamente enumerabili non sono decidibili, bensì solo semidecidibili: esiste un algoritmo che risponde affermativamente se $w \in L$, ma non fornisce alcuna risposta (ovvero non termina) se $w \notin L$. Esiste un più generale risultato

di indecidibilità che risale a Rice [1953], secondo cui le proprietà non banali degli insiemi ricorsivamente enumerabili (inclusi quindi i linguaggi riconosciuti da MdT) sono tutte indecidibili. In questo contesto, una proprietà non è banale se alcuni linguaggi della classe (ma non tutti) la soddisfano.

Concludendo, MdT e grammatiche generali fissano i confini della rappresentazione di linguaggi formali, mentre ASF (e automi a pila) segnano le basi per un approccio computazionale ai problemi interdisciplinari accennati all'inizio di questo lavoro.

6. NOTE CONCLUSIVE

Le numerose applicazioni tuttora attuali e la presenza di interessanti problemi aperti rendono la teoria degli automi un campo di ricerca vitale e prolifico. Esempi d'uso variano dalla elaborazione del linguaggio naturale (cfr. Roche e Schabes [1997]) ai compilatori per linguaggi di programmazione (cfr. Aho, Sethi, e Ullman [1986]) e alla biologia molecolare (cfr., Waterman [1995]). Per approfondire l'argomento senza cadere nei dettagli di specifici campi applicativi sono diversi i testi classici degni di nota. Tra i tanti, nel campo delle scienze informatiche due riferimenti chiari ed esaustivi sono Harrison [1978] e Hopcroft et al. [2007]. Sono più incentrati sulla teoria dei linguaggi formali ma decisamente più completi in questo contesto i contributi di Salomaa [1973] e [1981]. Per un approccio più filosofico, una interessante lettura è Novaes Dutilh [2012].

BIBLIOGRAFIA

Aho, A. V., Sethi, R., Ullman, J. D. (1986). *Compilers, principles, techniques and tools*. Addison Wesley.

- Backus, J. W. (1959). "The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM conference." In *International conference on information processing* (pp. 125–132).
- Berstel, J. (1995). *Axel Thue's papers on repetitions in words: a translation* (Publications du LaCIM, 20). Université du Québec à Montréal.
- Büchi, R. (1960). "On a decision method in restricted second order arithmetic." In *Congress on logic, method, and philosophy of science* (pp. 1–12). Stanford University Press.
- Chomsky, N. (1957). *Syntactic structures*. Mouton.
- Chomsky, N., Miller, G. (1958). "Finite state languages." *Information and Computation*, 1, 91–112.
- Chomsky, N., Schützenberger, M. P. (1963). "The algebraic theory of context-free languages." In P. Braffort D. Hirschberg (Eds.), *Computer programming and formal systems* (pp. 118–161). North Holland.
- Church, A. (1936). "An unsolvable problem of elementary number theory." *American journal of mathematics*, 58, 345–363.
- Davis, M. (1985). *Computability and unsolvability*. Dover Publications.
- Frege, G. (1893). *Grundgesetze der Arithmetik*. Verlag Hermann Pohle. (Tr. inglese di: M. Furth (1964), *The Basics Laws of Arithmetic: Exposition of the System*, Berkeley University Press; P. Ebert, M. Rossberg e C. Wright (2013), *Gottlob Frege: Basic Laws of Arithmetic*, Oxford University Press.)
- Ginsburg, S., Greibach, S., Harrison, M. (1967). "Stack automata and compiling."

Journal of the ACM, 14(1).

Gödel, K. (1931). “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme.” *Monatshefte für Mathematik und Physik*, 38. (Tr. in. a cura di S. Feferman (1986), *On Formally Undecidable Propositions of Principia Mathematica and Related Systems*, Oxford University Press.)

Greibach, S. (1981). “Formal languages: Origins and directions.” *Annals History of Computation*, 3.

Harrison, M. A. (1978). *Introduction to formal language theory*. Addison Wesley.

Hilbert, D. (1928). “Die Grundlagen der Mathematik.” In *Abhandlungen aus dem Seminar der Hamburgischen Universität*, 6. (Tr. in. a cura di J. van Heijenoort (1967), *From Frege to Gödel. A Source Book in Mathematical Logic, 1897–1931*, Harvard University Press.)

Hopcroft, J. E., Motwani, R., Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation*. Addison Wesley.

Immerman, N. (1988). “Nondeterministic space is closed under complementation.” *SIAM Journal of Computing*, 17, 275–303.

Jiang, T., Li, M., Ravikumar, B., Regan, K. W. (2010). “Formal grammars and languages.” In *Algorithms and theory of computation handbook*. Chapman & Hall, CRC Princeton University Press.

Kleene, S. (1951). *Representation of events in nerve nets and finite automata* (Research Memorandum No. RM-704). U.S. Air Force.

Kleene, S. (1956). “Representation of events in nerve nets and finite automata.” In

- C. Shannon J. McCarthy (Eds.), *Automata studies* (pp. 3–42). Princeton University Press.
- Kuroda, S.-Y. (1964). “Classes of languages and linear-bounded automata.” *Information and Control*, 7(2), 207–223.
- Landweber, P. (1963). “Three theorems on phrase structure grammars of type 1.” *Information and Control*, 6(2), 131–136.
- McCulloch, W., Pitts, W. (1943). “A logical calculus of the ideas immanent in nervous activity.” *Bulletin of Mathematical Biophysics*, 5, 115–133.
- McNaughton, R., Yamada, H. (1960). “Regular expressions and state graphs for automata.” *IEEE Transactions on Electronic Computers*, 9(1), 39–47.
- Myhill, J. (1960). *Linearly bounded automata* (Technical Note). Wright-Patterson Air Force Base.
- Novaes Dutilh, C. (2012). *Formal languages in logic: A philosophical and cognitive analysis*. Cambridge University Press.
- Post, E. L. (1936). “Finite combinatory processes.” *Journal of Symbolic Logic*, 1, 103–105.
- Post, E. L. (1946). “A variant of a recursively unsolvable problem.” *Bulletin of American Mathematics Society*, 52, 264–268.
- Post, E. L. (1947). “Recursive unsolvability of a problem of Thue.” *Journal of Symbolic Logic*, 12(1), 1–11.
- Pressburger, M. (1929). “Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt.” In *I con-*

- grés de mathématiciens des pays Slaves* (pp. 92–101). Warszawa. (Tr. in. di: R. Stanisfer (1984), *Pressburger's Article on Integer Arithmetic: Remarks and Translation*, Cornell University.)
- Rabin, M., Scott, D. (1959). "Finite automata and their decision problems." *IBM Journal of Research and Development*, 3(2), 114–125.
- Rice, H. G. (1953). "Classes of recursively enumerable sets and their decision problems." *Transactions of the American Mathematical Society*, 74(2), 358–366.
- Roche, E., Schabes, Y. (1997). *Finite-state language processing*. MIT Press.
- Salomaa, A. (1973). *Formal languages*. Academic Press.
- Salomaa, A. (1981). *Jewels of formal language theory*. Computer Science Press.
- Schützenberger, M. P. (1965). "On finite monoids having only trivial subgroups." *Information and Control*, 8, 190–194.
- Sénizergues, G. (1997). "The equivalence problem for deterministic pushdown automata is decidable." In *Automata, languages and programming* (Vol. LNCS 1256, pp. 671–681). Springer.
- Szelepcsényi, R. (1988). "The method of forced enumeration for nondeterministic automata." *Acta Informatica*, 26, 279–284.
- Thue, A. (1906). "Über unendliche Zeichenreihen." In *Norske Vid. Skrifter I Mat.-Nat. Kl.*, 7. Christiania.
- Thue, A. (1914). "Probleme über Veränderungen von Zeichenreihen nach gegebenen Regeln." In *Norske Vid. Skrifter I Mat.-Nat. Kl.*, 10. Christiania.
- Turing, A. (1937). "On computable numbers with an application to the

Entscheidungsproblem.” In *London mathematical society*, 42 (pp. 230–265).

Vardi, M., Wolper, P. (1986). “Automata-theoretic techniques for modal logics of programs.” *Journal of Computer and System Sciences*, 32(2), 183–221.

Waterman, M. S. (1995). *Introduction to computational biology*. Chapman and Hall.

APhEx.it è un periodico elettronico, registrazione n° ISSN 2036-9972. Il copyright degli articoli è libero. Chiunque può riprodurli. Unica condizione: mettere in evidenza che il testo riprodotto è tratto da www.aphex.it

Condizioni per riprodurre i materiali → Tutti i materiali, i dati e le informazioni pubblicati all'interno di questo sito web sono "no copyright", nel senso che possono essere riprodotti, modificati, distribuiti, trasmessi, ripubblicati o in altro modo utilizzati, in tutto o in parte, senza il preventivo consenso di APhEx.it, a condizione che tali utilizzazioni avvengano per finalità di uso personale, studio, ricerca o comunque non commerciali e che sia citata la fonte attraverso la seguente dicitura, impressa in caratteri ben visibili: "www.aphex.it". Ove i materiali, dati o informazioni siano utilizzati in forma digitale, la citazione della fonte dovrà essere effettuata in modo da consentire un collegamento ipertestuale (link) alla home page www.aphex.it o alla pagina dalla quale i materiali, dati o informazioni sono tratti. In ogni caso, dell'avvenuta riproduzione, in forma analogica o digitale, dei materiali tratti da www.aphex.it dovrà essere data tempestiva comunicazione al seguente indirizzo (redazione@aphex.it), allegando, laddove possibile, copia elettronica dell'articolo in cui i materiali sono stati riprodotti.

In caso di citazione su materiale cartaceo è possibile citare il materiale pubblicato su APhEx.it come una rivista cartacea, indicando il numero in cui è stato pubblicato l'articolo e l'anno di pubblicazione riportato anche nell'intestazione del pdf. Esempio: Autore, *Titolo*, «www.aphex.it», 1 (2010).